MELLOX AI

01

Product & Feature Workflows

What Mellox does, how people use it, and what each workflow needs to work.

REBUILD BLUEPRINT SERIES / VOLUME 01 OF 06

Source snapshot: 21 October 2026 | Branch: master

Commit: 055c90977c2c | Internal working reference

Code-derived implementation. Production activation requires separate evidence.

01 How to use this volume

This is volume 01 of six documents describing Mellox AI as represented in the local source snapshot on 21 October 2026. The owner confirmed that this repository is current, Railway is the hosting platform, codebase pricing should be used, and mellox.ai is the public website.

Evidence labels: Implemented means source exists and is mapped here; it does not prove that a production migration, credential, permission or feature switch is enabled. Owner-confirmed identifies facts supplied by the owner. External-verified identifies official web information checked on the snapshot date. Recommended identifies a rebuild or operating procedure. Unverified identifies missing deployment or commercial evidence.

Runtime code, migrations and current configuration outrank older guides. Model and price tables describe defaults unless explicitly stated otherwise. They are a dated reference, not a promise that vendors will retain them. No credentials or customer records are included.

Volume	Read it for
01 Product & workflows	Feature behavior, journeys, approvals and module ownership
02 Architecture & engineering	Frontend, backend, integrations, APIs and implementation patterns
03 AI models & generation	Every task-model route, images, video, context and evaluations
04 Pricing & billing	Plans, feature gates, all action prices, wallets and economics
05 Railway & operations	Deployment, configuration, jobs, security, recovery and diagnostics
06 Rebuild & data reference	Build order, acceptance criteria, full typed schema and migration index

Contents

01 How to use this volume 2

02 Product model and operating loop 4

03 Identity, onboarding and roles 4

04 Brain and Brand DNA 5

05 Team memory and chat 5

06 Studio, content library and calendar 6

07 Editorial lifecycle and approval integrity 7

08 Images, video ads and longer video 7

09 Stories and social publishing 7

10 Website publishing and creative integrations 8

11 Audience, competitors, market and Coach 8

12 Autopilot and agency Command Center 9

13 AI Visibility, fixes and tracked prompts 9

14 Proof Engine, analytics and backlinks 9

15 External assistant and staff surfaces 10

16 Commercial feature availability matrix 10

17 Future development and unresolved facts 11

18 Source ledger and verification limits 12

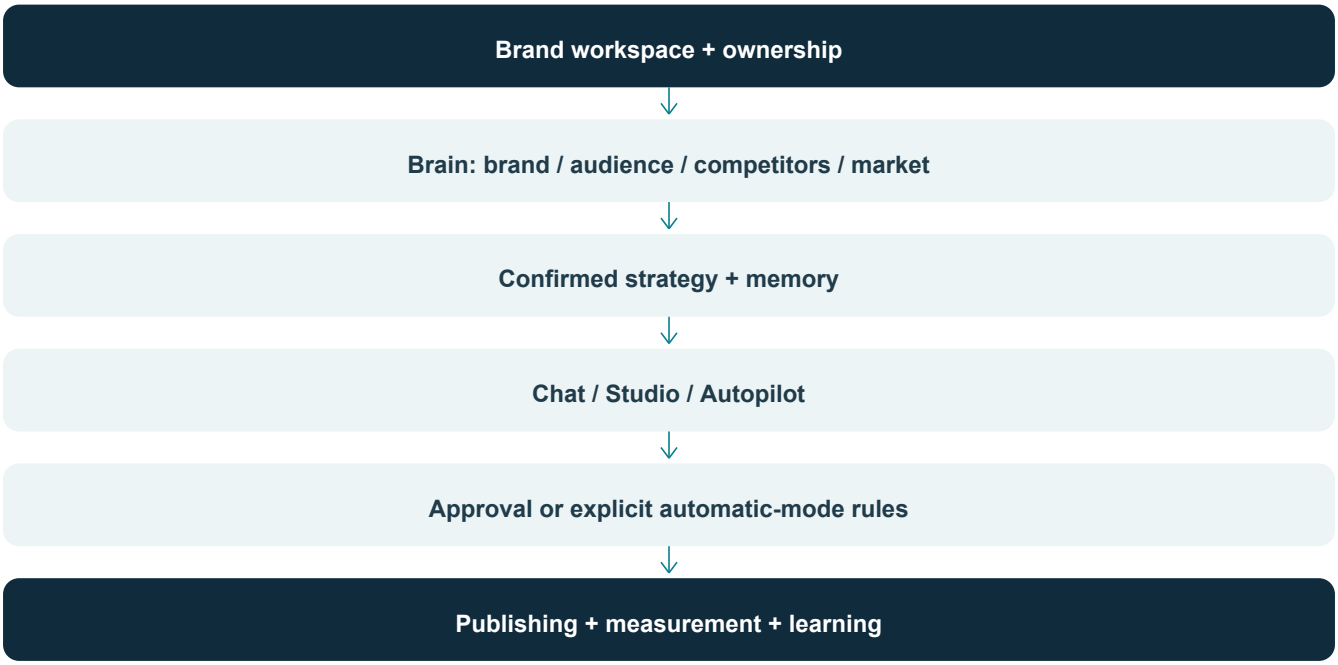
02 Product model and operating loop

Mellox AI is a workspace-based marketing product. It combines persistent brand knowledge, a marketing assistant, content and media production, social and website publishing, audience simulation, market and competitor research, analytics, and AI-search visibility improvement. Agencies can operate several client brands from one account.

The core loop is: establish the brand, understand the market and audience, choose a strategy, create work, review or explicitly configure automation, distribute, measure, and feed observations into the next decision. A workspace owns brand data and activity. Its owner billing account supplies shared allowances and balances across that owner's brands.

The product is not fully described by its marketing pages. A feature may have UI, code and catalog entitlement but still need a migration, provider account, permission or switch. Treat the current source catalog as commercial intent and trace execution through the server before promising availability.

Implementation sources: src/components/app; src/lib/billing/catalog.ts; src/server/workspaces/service.server.ts



03 Identity, onboarding and roles

People sign up or log in through Supabase Auth, then enter the projects/workspace selection flow. The selected brand is explicit in routes such as /w/<workspaceId>/app. Supabase identity proves who the person is; workspace membership and role decide which brand operations they may perform. Browser navigation state is not authorization.

Email confirmation, OAuth callbacks and password recovery have dedicated flows that capture URL credentials once and remove them before asynchronous work. Password reset must establish the recovery session before displaying a usable password form. Failed sign-out keeps the existing session visible and offers retry.

A rebuild should create the workspace boundary before building generators. Every query key and saved asynchronous result must retain its initiating workspace ID. Switching brands must discard old workspace queries so delayed responses cannot leak the previous client's data.

Implementation sources: src/server/workspaces/service.server.ts; src/lib/workspace; src/integrations/supabase/client.ts; src/lib/password-recovery.ts

Role	Intended boundary
Owner	Workspace lifecycle, owner billing and administrative authority; destructive actions have explicit checks.
Admin	Manage workspace settings and administrative features where the owning service permits.
Editor	Create and change content, start paid work and use authorized publishing or connector actions.
Viewer	Read authorized workspace data; cannot spend through the metered action path.
Platform staff	Separate admin permission system; workspace roles do not grant platform administration.

04 Brain and Brand DNA

Brain consolidates Brand, Audience, Competitors, Market and Strategy. Its home and Today views read existing information and show health, missing context, updates and a Coach briefing. Browser last-seen timestamps affect notification highlighting, not the authoritative knowledge record.

Brand DNA stores identity, positioning, audience, voice, logo, colors, typography, assets and rules. Website extraction and visual analysis populate context, but a person can correct it. Current Brand Look combines writing, visual and video preferences with DNA. A plain DNA record remains valid when no custom look exists. The older Brand Styles/Brand Kit design is superseded by the current Brain/Look implementation; do not rebuild that older feature as a second parallel source of style truth.

Strategy generation gathers the four brains, uses the premium text route, validates competitor and source references, then saves a draft. Editing is free. Only the person's confirmed strategy enters Studio, chat, Coach and Autopilot context. Changed source knowledge can mark a strategy stale; it must not silently regenerate and spend. The first strategy is included; subsequent rebuilds use the catalog's strategy_rebuild action.

Implementation sources: docs/brain.md; src/server/brain; src/server/strategy; src/lib/brand-look; src/server/brand-visuals

05 Team memory and chat

Team memory is distinct from conversation history. A memory is a rule, preference, fact or context sentence, categorized by topic. Current rules cap a brand at 200 memories, a reply at five changes, and temporary memory at one hour to seven days. Removed entries retain a marker so an old extraction does not immediately resurrect them. Members can read; edits need editor privileges; the workspace-wide switch and clear-all need admin.

Chat uses brand context, confirmed strategy, audience context, relevant memory, research and summarized history. Mellox Flash and Mellox Pro select different model routes; the internal model details are in volume 03. Read tools can inspect the workspace immediately. Change tools offer stored action buttons rather than applying the requested change during a model tool call. Memory operations are a deliberate exception: they can run immediately through the memory decision rules.

An action button executes the server-stored request after rechecking the clicking person's role. A compare-and-set moves offered to running, so repeated clicks cannot execute twice. A stale, invalid or already-used action must show a meaningful result. Read tool results are treated as data, not instructions. The reply metadata persists offered actions and memory notices so a reload does not lose the user's next step.

Implementation sources: docs/memory.md; src/server/chat; src/server/memory; src/lib/memory

06 Studio, content library and calendar

Studio turns a brief into social copy, an image post, carousel, Story, article, script, ad or video. It can also suggest ideas, improve a prompt, research an angle, caption uploaded material, revise an existing result, and create channel variants. Stored workspace context includes recent pieces, upcoming schedule, market evidence, competitor positioning, performance, audience, strategy and memory.

The request validates workspace, role, feature entitlement, balance and inputs before work begins. Long-running generation creates a studio_jobs record. A worker acquires a lease, creates or polls provider tasks, validates results, stores assets, and persists drafts. Generation success does not itself mean publication. Cancellation, provider failure, partial success and stale leases need explicit terminal handling and billing settlement.

The content library is the editorial store; assets are separate persisted media. The calendar displays planned slots and content dates, while delivery schedules use publishing operations. Calendar text saves are serialized per post; approving, scheduling or rewriting waits for those saves. An error preserves unsaved edits and blocks the dependent action. Moving a date does not bypass provider scheduling.

Carousels use layout/rendering and media services rather than requiring an AI image model for every graphical element. Keep the brand look, ordered slides, source references, caption and asset ownership together. The paid action table in volume 04 is the current charge reference; old guides that charge social publication credits are superseded.

Implementation sources: src/server/studio; src/lib/studio; src/server/fns/content.ts; src/server/fns/schedules.ts; src/components/app/StudioRail.tsx



07 Editorial lifecycle and approval integrity

Editorial status and delivery status are different authorities. Normal content writes accept draft, pending, approved and rejected. Scheduling and publication use their dedicated server paths. Scheduled and publishing posts are locked against ordinary editorial changes; cancel delivery first. Published content must return to draft before editorial reuse where required.

Writes compare the read status and updated_at revision. A conflicting save returns 409 rather than overwriting newer work. A kind change invalidates a prior approval through a database trigger. This prevents an approved text post becoming an approved video or Story without renewed review.

The publishing queue can return an incomplete result when an optional website source is unavailable. UI must retain valid results and label the missing source rather than show an apparently complete empty queue. Reconciliation records provider truth; a browser cannot directly mark something published.

Implementation sources: src/server/fns/content.ts; src/server/fns/publish.ts; supabase/migrations/20261018090000_content_kind_approval.sql

08 Images, video ads and longer video

Images support generation, reference-led editing and variations. The image router selects a fast or precision model using the brief, complexity, quality, edit/reference needs and available models. Persisted assets belong to the workspace and use private storage. Image-generation text and visual safety checks are server-owned.

UGC video starts with product extraction or supplied product information and images. Concept packs, notes and scripts lead to a plan-eligible model, duration, aspect ratio, resolution and reference mode. The server quotes video allowance from the routed provider's cost and current catalog, places a hold, submits, polls, checks safety, persists the finished output, then settles the hold. A finished render can become a content draft.

The current duration options include 10, 15, 20, 30, 40 and 60 seconds; provider-native durations also remain supported. Longer output is composed from provider-sized clips. Durable video_compositions records store accepted tasks across polls and restarts. FFmpeg trims padding and assembles the result. This is multiple clips, not evidence that the upstream model can natively create a continuous sixty-second take. Creator, voice and product continuity depend on generation quality and require review.

Implementation sources: src/lib/model-router.server.ts; src/lib/openrouter-image.server.ts; src/lib/ugc/duration.ts; src/server/ugc/providers/composition.server.ts; src/server/ugc/providers/assemble.server.ts

09 Stories and social publishing

A Story is a content_items row with kind story, not a separate editorial queue. Frames retain their order. Studio can produce 9:16 image frames or reuse an eligible vertical video. Current Studio billing describes up to seven frames; Autopilot Story settings use one to five frames. Instagram and Facebook are the Story destinations represented by the integration.

Mellox's distribution catalog exposes LinkedIn, X, Instagram, Facebook, Threads, TikTok and YouTube. The provider may support additional networks; that does not automatically make them Mellox features. A destination's media, account type, privacy and other constraints are validated through the provider path. Current code prioritizes Post for Me when configured, supports optional self-hosted SDR, and explicitly retires SocialAPI.

Each Story frame becomes a separate provider Story. Results and partial failures are tracked by frame; retry must avoid resending successful frames and must refuse changed source media. Native music, link, poll, question and location stickers are unavailable in this connector. Analytics needs feed permissions, and missing metrics remain unavailable rather than estimated.

Social publishing does not consume general AI credits. It still has account authorization, approval, provider availability and a monthly anti-abuse fair-use cap. Scheduling success means provider acceptance or stored scheduling state, not proof that the network has already published it.

Implementation sources: docs/stories.md; src/lib/distribution-platforms.ts; src/lib/feature-flags.ts; src/lib/postforme; src/server/social

10 Website publishing and creative integrations

Article publishing is separate from social distribution. An editor previews the target website, host, blog readiness, slug and schedule, then approves the publication. GitHub writes produce a reviewable branch and pull request. WordPress and Webflow use their connected service adapters. A connected provider alone does not prove that a selected website supports every operation.

Canva is an explicit editing round trip. Mellox uploads workspace-owned media, creates or resolves a design, opens Canva, then imports the user's requested export as new versioned assets. Original assets remain. Imported edits can return an approved or pending post to draft; scheduled and published posts are not silently changed. Preview Magic Layers can be disabled while the flat-image path works. A carousel can map to several Canva designs in slide order.

Notion is a manual planning/export/import surface. Mellox remains authoritative. Imports preview changes and create or update eligible drafts; Notion status cannot publish or schedule them. One-sided changes can sync; two-sided changes are conflicts. Slack is an opt-in connector with dedicated inbound processing and workspace binding. Its exact enabled production installation was not inspected.

Implementation sources: src/server/articles; src/server/fns/site-publishing.ts; docs/canva-integration.md; docs/notion-integration.md; src/server/slack; src/server/fns/slack.ts

11 Audience, competitors, market and Coach

Audience builds sourced customer groups, predicts response to exact content, explains the dimensions, simulates audience reactions, compares variants and learns from later outcomes. Predictions are not real customer interviews. Each statement keeps provenance and guesses remain labeled. A person's edited group is protected from rebuild overwrite. A cached prediction matches subject_hash, twins_fingerprint and score_version; changed content cannot retain an old score as current.

Competitor discovery produces suggestions; tracking is a separate choice. Profiles, relevant updates and periodic sweeps feed intelligence. Market Brain stores dated sources, trends, opportunities and recommendations. Search and extraction providers gather evidence; the model classifies or synthesizes it. A provider error must not become evidence that a rival did nothing or that the brand was absent.

Coach turns stored brand and performance context into prioritized next actions. Cached briefings and scheduled included briefings differ from paid on-demand or deep strategy actions. Brain health indicates completeness of stored information, not independently proven business quality. Analytics and market sources can be unavailable; the product must preserve their provenance and freshness.

Implementation sources: docs/audience.md; src/server/audience; src/server/research; src/lib/market-brain-latest.server.ts; src/server/fns/coach.ts; src/server/fns/competitors.ts

12 Autopilot and agency Command Center

Autopilot proposes a program and weekly plan, makes content through Studio, coordinates review, schedules eligible pieces and records later outcomes. The three approval modes are fully automatic, approve each post, and approve the plan plus each post. Current automatic-mode rules can send qualifying plain and image posts without a per-post click after the user enables that mode; other content waits. Fully automatic mode limits automatic feed sending to two posts per day. Opportunity-created pieces always wait for approval.

Plans rotate the brand's themes and content purposes, respect channel formats and existing calendar slots, and attach grounded audience, market and competitor context. A piece generally generates three days before its slot. Weekly general-credit and video limits skip work that would exceed budget. Missed approval windows and stale Stories must not publish late as though they were current.

Additional automations include weekly AI visibility checks, reusing qualifying measured posts, optional website article delivery, summary email and learning. Website article publishing still requires the person's approval. Summary emails read stored results; they are not a new generated strategy. Email availability depends on configured delivery settings.

Autopilot lives in the chat composer and a dedicated workspace route rather than requiring a permanent sidebar item. Agency Command Center summarizes client workspaces and approval/health activity. Cross-client views must retain each workspace's membership boundary and identify failures rather than lose them in totals.

Implementation sources: docs/autopilot.md; src/lib/autopilot; src/server/autopilot; src/components/app/command-center

13 AI Visibility, fixes and tracked prompts

The website scanner fetches allowed public pages, analyzes technical and content evidence, applies deterministic rules, and stores scores, findings and explanations. Quick scans and durable full-site scans have different execution paths. GEO, AEO and SEO are related dimensions in this product; a high technical score is not proof of ranking or AI citation lift.

Tracked prompts ask answer-engine models and record brand mentions and citations over time. Mention and citation are different measurements. API answer probes are observations of that API/model, not a guarantee of identical responses in every consumer ChatGPT, Gemini or Perplexity interface. Provider failures must be separate from negative visibility observations.

Fixes can include deterministic guidance, generated proposals, batches, CMS actions or a repository-scoped engineer. Source ownership, reviewed base revision, exact patch approval and server-side verification constrain changes. Approval and a merged pull request do not by themselves resolve a finding: a verification scan must observe the correction. Dismissal is workflow state, not evidence of a repaired website.

Implementation sources: src/lib/geo; src/server/geo; src/server/fns/tracked-prompts.ts; src/server/connectors/github/ownership.server.ts

14 Proof Engine, analytics and backlinks

Proof Engine creates controlled website experiments with treatment/control pages. Its v1 GitHub integration uses a data-file override and reviewed PRs; it does not simply declare uplift from a before/after chart. Dedicated page-level Google pulls preserve low-traffic pages that dashboard top-N sync would omit. Current analysis writes daily early readings and evaluates final checkpoints after 21, 28, 35 and 42 complete post-live data days. Insufficient or invalid data must stay inconclusive.

Analytics connects GA4, Search Console and available social metrics, stores sync state and daily data, computes aggregates and can generate interpretations. Product definitions such as attribution, reporting timezone and contractual reporting guarantees still need owner-level decisions; code field names alone do not establish those definitions.

Backlink Growth has a supplier catalog and matching, profiles, orders, reconciliation and live page verification. This subsystem also has separate link-purchase credit and Stripe code. Do not merge link placement purchasing with the owner's AI-generation wallet merely because both use the word credits. Provider contract and placement inventory availability remain external dependencies.

Implementation sources: src/server/experiments; src/lib/experiments; src/server/analytics; src/server/links; src/server/backlinks; src/server/billing/stripe.server.ts

15 External assistant and staff surfaces

The MCP endpoint lets an authenticated external assistant use the app's own server operations. An admin enables assistant access per workspace and separately enables changes. Role, plan, balance, rate and approval checks still apply. Billing changes, team administration, destructive workspace deletion and applying website patches are deliberately outside this exposed tool set. MCP exposure is not the coming-soon commercial SSO/API feature.

The source also includes a separate staff admin console: users, workspaces, entitlements/credits, usage, jobs, health, flags, AI controls, announcements, abuse, audit and staff access. Active admin_users entries use explicit permissions; an existing environment allowlist can provide break-glass super-admin access. MFA is required by default in production. Dangerous actions can demand a recent second-factor check. This is internal operational functionality, not an ordinary customer role.

Implementation sources: src/server/mcp; docs/mcp.md; src/server/admin; src/lib/admin; src/server/platform

16 Commercial feature availability matrix

This is extracted from the current catalog. Launch indicates catalog availability, not proof of configured production readiness. Runtime switches and permissions can narrow it.

Feature	Minimum plan	Catalog status
Studio	free	launch
Premium articles	starter	launch
Mellox Pro chat	starter	launch
UGC video ads	starter	launch
1080p video	growth	launch
Premium and Long-take video	growth	launch
Cinematic video	agency	launch
Campaign plans	growth	launch
Approval workflows	growth	launch

Feature	Minimum plan	Catalog status
GEO fix proposals	starter	launch
One-click fixes	growth	launch
GEO Engineer and Fix all	growth	launch
Proof Engine experiments	starter	launch
Competitor tracking	starter	launch
Market Brain	starter	launch
Audience	starter	launch
GA4 and Search Console insights	starter	launch
Publishing	free	launch
Backlink marketplace	growth	launch
Client portal	growth	launch
Agency command center	agency	launch
Autopilot	growth	launch
White-label	agency	launch
SSO and API access	scale	later

Implementation sources: src/lib/billing/catalog.ts

17 Future development and unresolved facts

The catalog explicitly marks SSO/API access, daily prompt tracking, daily Market Brain, Google AI Overviews prompt add-on and custom white-label domains as later. They belong in the development backlog, not the current shipped feature list. Existing MCP tools do not establish that the commercial SSO/API product is complete.

Highest-priority documentation decisions still require operational evidence: production service inventory, deployment commit, migration application state, enabled switches, provider app approvals, retention and backup policy, metric definitions, contractual SLA and actual invoice-based cost to serve. This series records the known boundaries instead of inventing those facts.

Use volume 06 when implementing from scratch. It orders the build around tenant isolation and durable accounting first, then knowledge, generation, publication, measurement and automation. Each later module depends on those early foundations.

Implementation sources: src/lib/billing/catalog.ts; docs/documentation-maintenance.md

18 Source ledger and verification limits

Repository-relative paths below identify the owning implementation. The companion source folder includes an extracted inventory and SHA-256 hashes for source files and migrations. The inventory is a static extraction, not a database dump or a successful production test. Generated database types describe their checked-in state; migrations define actual SQL constraints, defaults and policies.

The Mellox connector failed with an internal error. The public site could not be fetched by the web tool or local network, and browser initialization was unavailable. Consequently public-site rendering, deployed commit, enabled credentials, vendor approvals, production schema, invoices, real usage and exact Railway service topology were not independently inspected. The local marketing implementation was used as a secondary source. No live provider generation or publication was triggered.

Source	SHA-256 prefix
src/components/app	directory reference
src/lib/billing/catalog.ts	eb845ac041f58b84
src/server/workspaces/service.server.ts	e1e5ddb7053563bc
src/lib/workspace	directory reference
src/integrations/supabase/client.ts	3d76962d05c21248
src/lib/password-recovery.ts	1495ac70af06af14
docs/brain.md	7a123b758803584a
src/server/brain	directory reference
src/server/strategy	directory reference
src/lib/brand-look	directory reference
src/server/brand-visuals	directory reference
docs/memory.md	c61e3ab06c3c27fd
src/server/chat	directory reference
src/server/memory	directory reference
src/lib/memory	directory reference
src/server/studio	directory reference
src/lib/studio	directory reference
src/server/fns/content.ts	cbd7e21890647951
src/server/fns/schedules.ts	4085644f05dc552b
src/components/app/StudioRail.tsx	43697beaa4b6fec3
src/server/fns/publish.ts	ba13a497e8368272
supabase/migrations/20261018090000_content_kind_approval.sql	df81942beba55d0e

Source	SHA-256 prefix
src/lib/model-router.server.ts	4c27a7b0f199930b
src/lib/openrouter-image.server.ts	84d3de04136c7acc
src/lib/ugc/duration.ts	6ad312021cd6c5d5
src/server/ugc/providers/composition.server.ts	4b92cd5607c286fb
src/server/ugc/providers/assemble.server.ts	b7903e3b13f6a114
docs/stories.md	c02991052c850119
src/lib/distribution-platforms.ts	76ecc11d2fd7a7d7
src/lib/feature-flags.ts	e56ac35e17629593
src/lib/postforme	directory reference
src/server/social	directory reference
src/server/articles	directory reference
src/server/fns/site-publishing.ts	be70a13643f0ca03
docs/canva-integration.md	3a4e314a5a82741a
docs/notion-integration.md	d34c7c9352c3f010
src/server/slack	directory reference
src/server/fns/slack.ts	1ecbf0d9e1f01786
docs/audience.md	c4ac9d67e5f3e11a
src/server/audience	directory reference
src/server/research	directory reference
src/lib/market-brain-latest.server.ts	353a6b7b36ec3214
src/server/fns/coach.ts	68f11d6d94766e05
src/server/fns/competitors.ts	81e2384d5d986780
docs/autopilot.md	c78829049dab54bb
src/lib/autopilot	directory reference
src/server/autopilot	directory reference
src/components/app/command-center	directory reference
src/lib/geo	directory reference
src/server/geo	directory reference
src/server/fns/tracked-prompts.ts	88b2714f26e26c59
src/server/connectors/github/ownership.server.ts	583710ca425837a5

Source	SHA-256 prefix
src/server/experiments	directory reference
src/lib/experiments	directory reference
src/server/analytics	directory reference
src/server/links	directory reference
src/server/backlinks	directory reference
src/server/billing/stripe.server.ts	e7256f848d31ea58
src/server/mcp	directory reference
docs/mcp.md	1c25c5f7cc14c32f
src/server/admin	directory reference
src/lib/admin	directory reference
src/server/platform	directory reference
docs/documentation-maintenance.md	8117afd85c931614